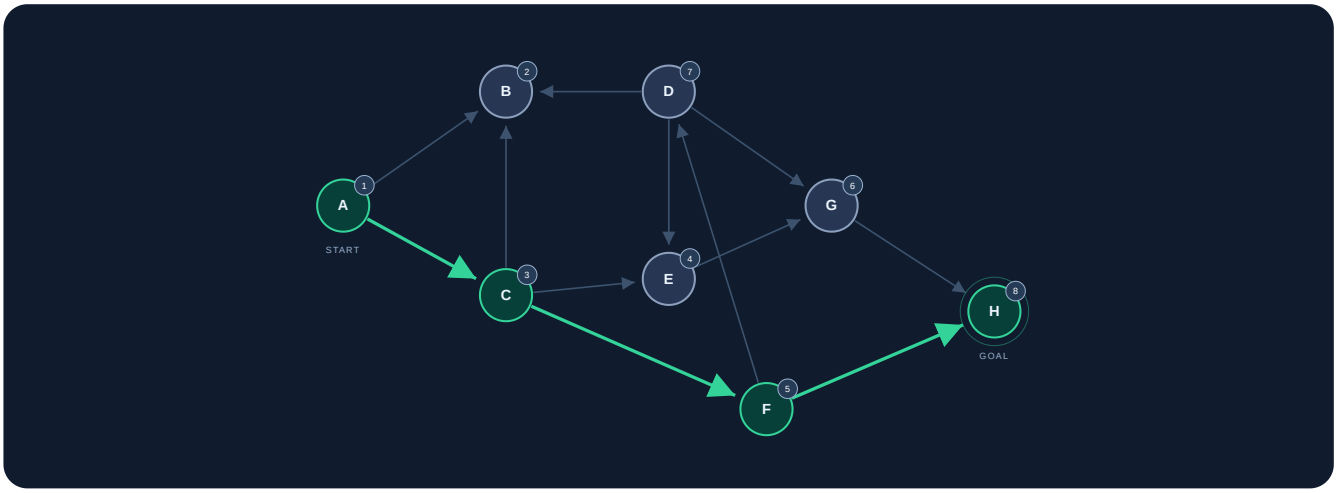
01 / THE PROBLEM

البحث بالعرض — BFS

نريد الوصول من A إلى H عبر الجراف الموجّه في المثال المحلول. كل وصلة تكلفتها واحدة، لذلك نبحث عن طريق بأقل عدد انتقالات.



الجراف المستخدم في التطبيق. المسار الأخضر هو الحل الذي تعيده BFS.

Problem Formulation

قبل اختيار الخوارزمية نحدد شكل المسألة. العناصر الستة التالية تصف ما يمكن فعله وما الذي نعتبره حلًا.

State space

الحالات الممكن الوصول لها من A: كل الحالات من A إلى H في هذا المثال.

Initial state

الحالة A هي نقطة البداية.

Actions

الانتقال إلى جار مباشر في اتجاه السهم. من A نختار B أو C.

Transition model

الحركة «انتقل إلى C» من A تجعل الحالة الجديدة C.

Goal test

هل الحالة الحالية تساوي H؟

Path cost

كل انتقال تكلفته 1؛ تكلفة المسار هي عدد الوصلات.

البداية A والهدف H هما افتراضا المثال المحلول المرفق. الوصلة بين F وD مقروءة D → F، والجيران يؤخذون بترتيب القوائم.

02 / QUEUE & TRACE

كيف تمشي الخوارزمية؟

BFS تفحص البداية، ثم جيرانها، ثم الحالات على المستوى التالي. أداة التنظيم هي Queue: أول عنصر يدخل هو أول عنصر يخرج، وهو ترتيب FIFO.

1. نضع البداية في الطابور ونسجّل أنها اكتُشفت.
2. نُخرج أول حالة من الطابور ونفحص هل هي الهدف.
3. إذا لم تكن الهدف، نضيف جيرانها غير المكتشفين إلى نهاية الطابور.
4. نكرر حتى نصل للهدف أو يفرغ الطابور.

تتبع المثال من A إلى H

الطابور يبدأ بـ [A]. الجدول يوضح محتواه بعد كل خطوة؛ المقدمة ناحية اليسار.

Step	Current	Added	Queue: front → back
1	A	B, C	[B, C]
2	B	—	[C]
3	C	E, F	[E, F]
4	E	G	[F, G]
5	F	D, H	[G, D, H]
6	G	—	[D, H]
7	D	—	[H]
8	H	—	[]

ترتيب الزيارة

A → B → C → E → F → G → D → H

مسار الحل

A → C → F → H

ترتيب الزيارة يشمل كل الحالات التي فحصناها. مسار الحل يضم الحالات المتصلة في الطريق إلى الهدف فقط؛ لذلك لا ننسخ ترتيب الزيارة باعتباره حلًا.

طول الحل 3 وصلات، وتكلفته 3. فُحصت 8 حالات، وأكبر حجم وصل إليه الطابور كان 3 حالات. فحص الهدف يتم وقت إخراجه من الطابور.

03 / STATES, ACTIONS & NODES

من الصياغة إلى Python

في هذا التمثيل، اسم الحركة هو اسم الوجهة. مثلًا الحركة "C" تعني «انتقل إلى C». تُختار الحركة من قائمة الجيران، ثم يعيد نموذج الانتقال الحالة الجديدة.

```
from dataclasses import dataclass

@dataclass
class SearchProblem:
    graph: dict[str, list[str]]
    initial_state: str
    goal_state: str

    @property
    def state_space(self):
        return reachable_states(self.graph, self.initial_state)

    def actions(self, state):
        return self.graph[state]

    def result(self, state, action):
        # action is selected from actions(state)
        return action

    def goal_test(self, state):
        return state == self.goal_state

    def step_cost(self, state, action):
        return 1
```

مقتطف من bfs.py مع حذف التحقق من المدخلات للاختصار. الدالة reachable_states الموجودة في الملف تجمع الحالات الممكن الوصول لها من البداية.

الحالة ليست عقدة البحث

State هي الحالة نفسها، مثل C. أما Node فتحفظ معها كيف وصلنا: الأب والحركة والتكلفة والعمق.

```
@dataclass(frozen=True)
class Node:
    state: str
    parent: "Node | None" = None
    action: str | None = None
    path_cost: int = 0
    depth: int = 0
```

في الحل، عقدة F أبُوها C، وعمقها 2، وتكلفة الوصول لها 2. روابط الآباء هي التي نستخدمها لاستخراج المسار.

04 / THE ALGORITHM

كود BFS الأساسي

النسخة القصيرة التالية موجودة في bfs_simple.py. تستخدم نفس الصياغة والعقد التي تستخدمها الواجهة، وتعيد مسار الحل وترتيب الزيارة.

```
from collections import deque
from bfs import Node, SearchProblem, build_path

def bfs_simple(problem: SearchProblem):
    queue = deque([Node(problem.initial_state)])
    discovered = {problem.initial_state}
    visit_order = []

    while queue:
        current = queue.popleft()
        visit_order.append(current.state)

        if problem.goal_test(current.state):
            return build_path(current), visit_order

        for action in problem.actions(current.state):
            next_state = problem.result(current.state, action)
            if next_state not in discovered:
                discovered.add(next_state)
                child = Node(
                    state=next_state,
                    parent=current,
                    action=f"Move to {next_state}",
                    path_cost=current.path_cost + 1,
                    depth=current.depth + 1,
                )
                queue.append(child)

    return [], visit_order
```

اكتشاف الحالة

نضيفها إلى discovered وقت دخولها الطابور، حتى لا تُضاف مرتين أو تكرر الدورة نفسها.

استرجاع الحل

build_path تتبع الآباء من الهدف للبداية، ثم تعكس القائمة ليظهر الطريق بترتيبه الصحيح.

تجربة المثال

```
python bfs_simple.py

Visit order: A -> B -> C -> E -> F -> G -> D -> H
Solution: A -> C -> F -> H
Path cost: 3
```

نسخة الأنيميشن في bfs.py تضيف حفظ اللقطات ومقاييس التنفيذ إلى خطوات البحث نفسها.

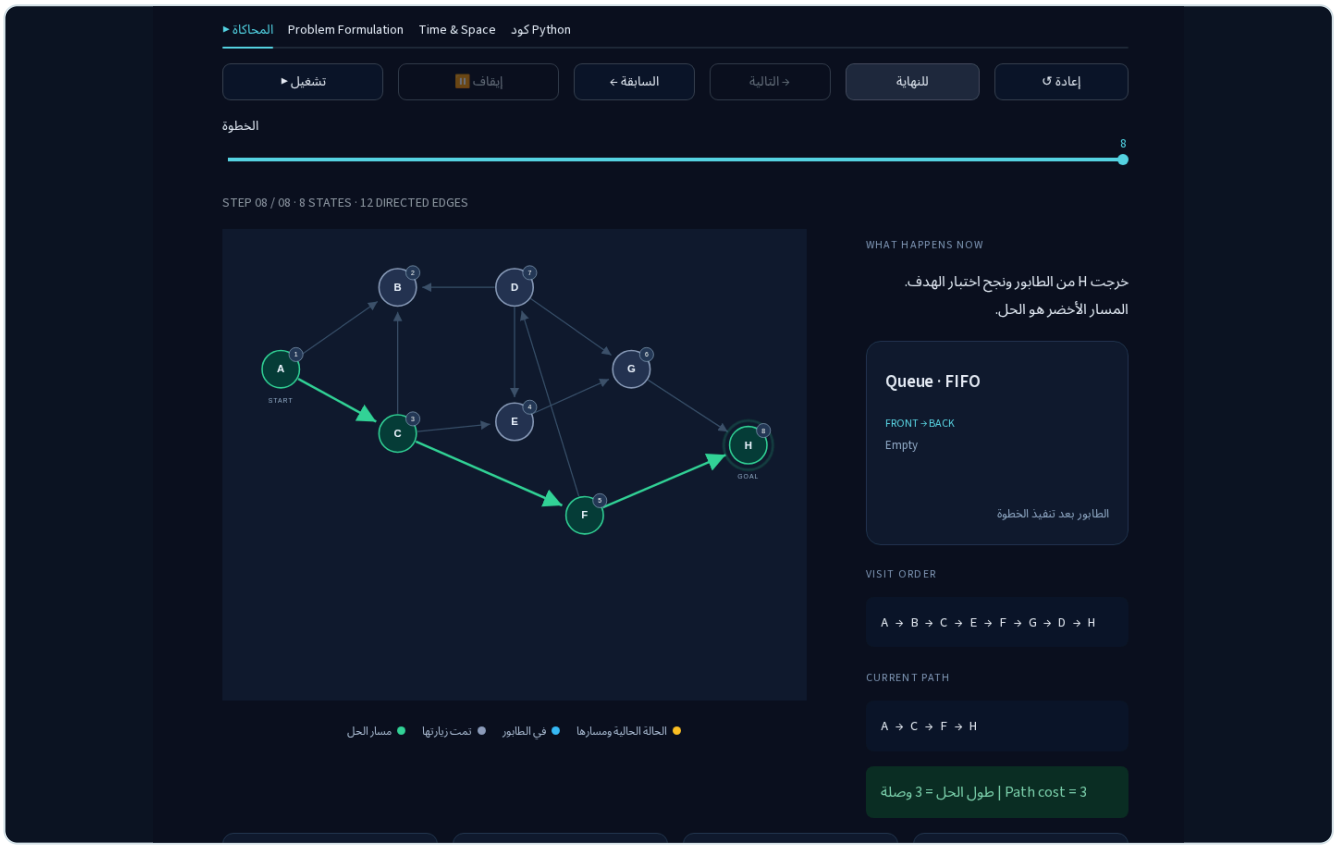
05 / STREAMLIT

تشغيل الواجهة ومتابعة البحث

افتح الطرفية داخل مجلد المشروع بعد فك الضغط. يتطلب التشغيل Python 3.11+، وتثبيت Streamlit يحتاج اتصال إنترنت في المرة الأولى.

```
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

افتح العنوان الذي يظهر في الطرفية، وغالبًا http://localhost:8501.



لقطة من التطبيق عند الوصول إلى H. المسار الأخضر هو الحل، والأرقام الصغيرة ترتيب الزيارة.

المحاكاة

شغّل الأنيميشن أو تحرك خطوة بخطوة. تابع الطابور، والحالة الحالية، وترتيب الزيارة. يمكن تغيير سرعة العرض وإعادة التجربة.

صياغة المسألة

حدد حالة وجرب حركة متاحة منها؛ سترى الحالة الجديدة والتكلفة ونتيجة اختبار الهدف. تتحدث العناصر الستة حسب المثال والبداية والهدف.

القائمة الجانبية تضم الجراف المحلول وشجرة السكشن ومثال المدن من ملف الصياغة. يمكنك إضافة جرافك بصيغة JSON، ثم تنزيل جدول التتبع بصيغة CSV.

> ابدأ قراءة المشروع من bfs_simple.py ثم bfs.py. الواجهة في app.py، والرسم في visuals.py، والأمثلة في examples.py.

06 / TIME & SPACE

الوقت والذاكرة وحدود الحل

تحليل الجراف المستخدم في الكود

العنصر	التعقيد	السبب
الوقت	O(V + E)	كل حالة تدخل الطابور مرة، ونفحص قوائم الجيران مرة.
الذاكرة الإضافية	O(V)	الطابور، والحالات المكتشفة، والعقد وروابط الآباء.

V عدد الحالات وE عدد الوصلات الموجّهة. هذه حدود BFS الأساسية دون تخزين الجراف أو سجل الأنيميشن.

تحليل شجرة البحث في السكشن

السكشن يذكر O(b^d) للوقت والذاكرة، حيث b أقصى عدد جيران وd عمق أقرب هدف. هذه الصيغة ترتبط بحساب المستويات حتى عمق الهدف أو التوقف عند توليده.

في تطبيقنا نختبر الهدف عند إخراجه من الطابور، مثل الملف المحلول. بعض حالات المستوى d قد تولّد جيرانًا من d+1 قبل خروج الهدف؛ لذلك الحد المحافظ في تحليل الشجرة هو O(b^(d+1)). أما حد الجراف المحدود فيظل O(V + E).

ماذا تقيس الواجهة؟

الوقت الظاهر يشمل البحث وحفظ لقطات التتبع، ولا يشمل رسم الواجهة أو الانتظار بين الخطوات. أكبر حجم للطابور هو عدد حالات، وليس قياسًا للذاكرة بالبايت.

> كل لقطة تنسخ الطابور وقوائم الاكتشاف والزيارة والمسار. لعدد S من اللقطات، تكلفة السجل O(S·V)، وزمن التطبيق O(V + E + S·V). لأن S ≤ V + 2، قد يصل سجل العرض إلى O(V²).

متى تكون النتيجة مضمونة؟

BFS كاملة في الجراف المحدود مع منع التكرار. وفي فضاء غير محدود يلزم تفرع محدود وهدف على عمق محدود. تضمن أقل عدد وصلات؛ وهذا يساوي أقل تكلفة في المسألة الحالية لأن كل انتقال تكلفته 1. اختلاف الأوزان يلغي ضمان أقل تكلفة.

إذا كانت البداية هي الهدف يرجع المسار بحالة واحدة وتكلفة صفر. وإذا كان الهدف غير قابل للوصول، يفرغ الطابور وتظهر رسالة مناسبة. الاختبارات تشمل هذه الحالات، والدورات، وتطابق المثال المحلول، وفحص أقصر المسارات على جرافات عشوائية.

المراجع

- BFS_DFS_Solved_Problem.pdf — directed graph, queue trace and solution.
- BFS&DFS section.pdf — BFS explanation, tree example and properties.
- Problem_Formulation.pdf — formulation components, states and nodes.